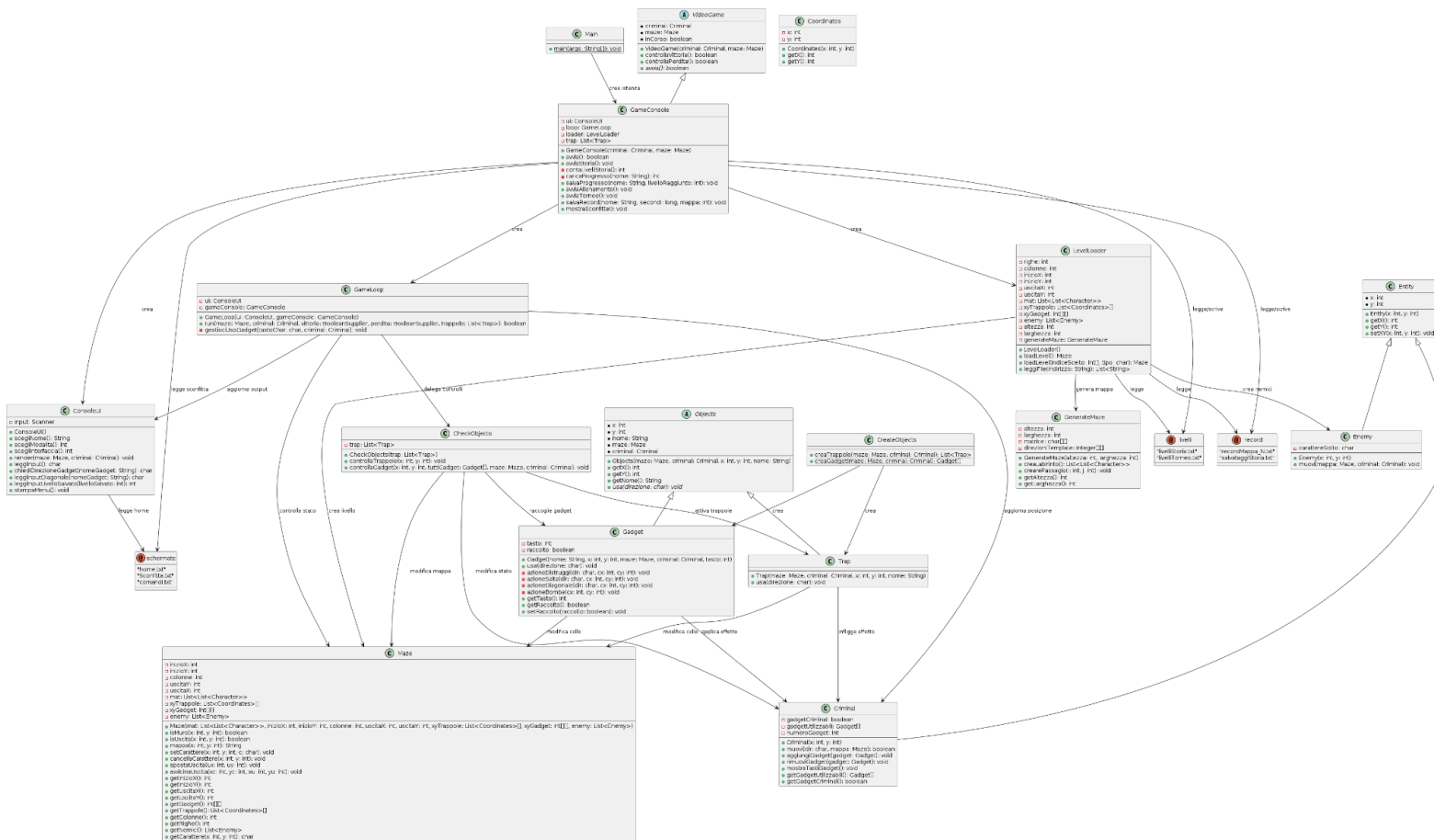


By Marco Caria e Lotan Teny

Java Doc:

[JavaDoc](#)

Schema UML:



codice consultabile al:

[PlantUML](#)

Le nostre scelte progettuali:

Architettura Model-View-Controller (MVC)

La scelta principale è stata la suddivisione del codice in tre pacchetti logici distinti per separare le responsabilità:

- **Model (it.volta.smoothcriminal.model)**: Contiene le classi che rappresentano i dati e le regole del mondo di gioco, come **Maze**, **Criminal** ed **Enemy**. Queste classi non sanno come verranno visualizzate, così che la logica di gioco sia indipendente dall'interfaccia.

- **View** ([it.volta.smoothcriminal.console](#)): Gestisce esclusivamente l'interazione con l'utente e la stampa a video.
- **Controller** ([it.volta.smoothcriminal.core](#)): Coordinano le azioni, leggendo l'input dalla View e aggiornando lo stato del Model.

Ereditarietà e Polimorfismo

Per ottimizzare il codice e ridurre la ridondanza, è stata utilizzata una gerarchia di classi basata su astrazione:

- **Classe Entity**: Centralizza la gestione delle coordinate X e Y per tutte le entità, permettendo a [Criminal](#) ed [Enemy](#) di ereditare i metodi di posizionamento senza duplicazioni.
- **Classe Astratta Objects**: Il metodo [usa\(char direzione\)](#) permette al sistema di gestire in modo polimorfico sia i [Gadget](#) che le [Trap](#). Il controllore può invocare l'azione di un oggetto calpestato dal giocatore senza conoscerne la fonte, delegando l'esecuzione alla sottoclasse corretta.

Gestione delle Risorse

Il sistema di caricamento è stato progettato per essere flessibile, ampliabile e facile da leggere/modificare:

- **LevelLoader**: Legge mappe in formato ASCII da file esterni ([.txt](#)). L'uso del delimitatore "___" permette di memorizzare più livelli in un unico file, facilitando l'aggiunta di nuovi contenuti senza modificare il codice Java.
- **Record e 'database'**: La scelta di salvare i progressi della modalità Storia e i record del Torneo su file separati consente di mantenere i dati dell'utente tra diverse sessioni di gioco.

Algoritmi

- **Generazione Procedurale**: La classe [GenerateMaze](#) implementa un algoritmo di *Recursive Backtracking* (DFS randomizzata). Questa scelta permette la creazione di "labirinti perfetti", ovvero mappe senza aree isolate e con un unico percorso risolutivo, assicurando che ogni partita in modalità Allenamento sia diversa ma giocabile.
- **Nemici**: La classe [Enemy](#) adotta una logica di inseguimento. Il nemico valuta costantemente la distanza relativa sull'asse X e Y rispetto al [Criminal](#), cercando di ridurre lo scarto maggiore, pur rispettando i vincoli di collisione dei muri del [Maze](#).

Software

- **IntelliJ**: Usata maggiormente per la possibilità di poter programmare contemporaneamente sullo stesso progetto da più utenti da pc diversi e per la compatibilità con GitHub dove era presente la repository.
- **GitHub**: La repository smoothcriminals, ora pubblica, permette di aggiornare il codice e di renderlo disponibile per più utenti, è presente anche un 'diario di bordo' che traccia lo sviluppo del gioco per tutta la durata dello sviluppo del gioco, ad oggi conta 31 commits!! (ovvero delle modifiche salvate sul codice)
- **PlantUML**: software stupendo su cui son presenti più di 264 linee di codice per lo schema presente prima!

Screenshot del gioco in esecuzione:

```
Seleziona la modalit  di visualizzazione:
1) Giocare su Console
2) Giocare su GUI (Interfaccia Grafica)
Scelta:
```

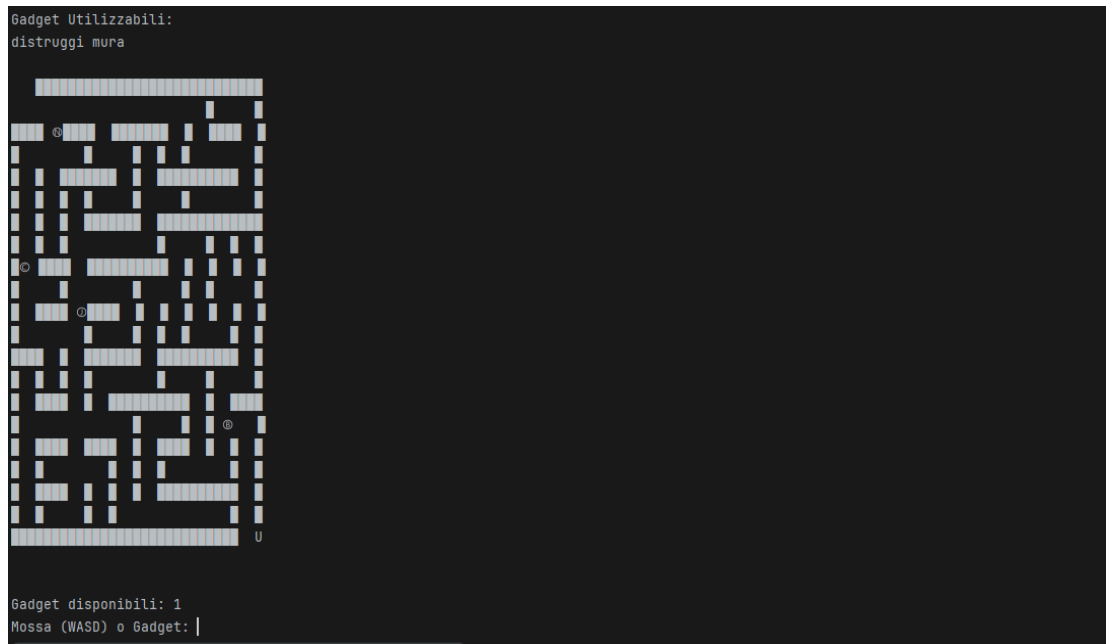
- scelta dell'interfaccia su cui giocare



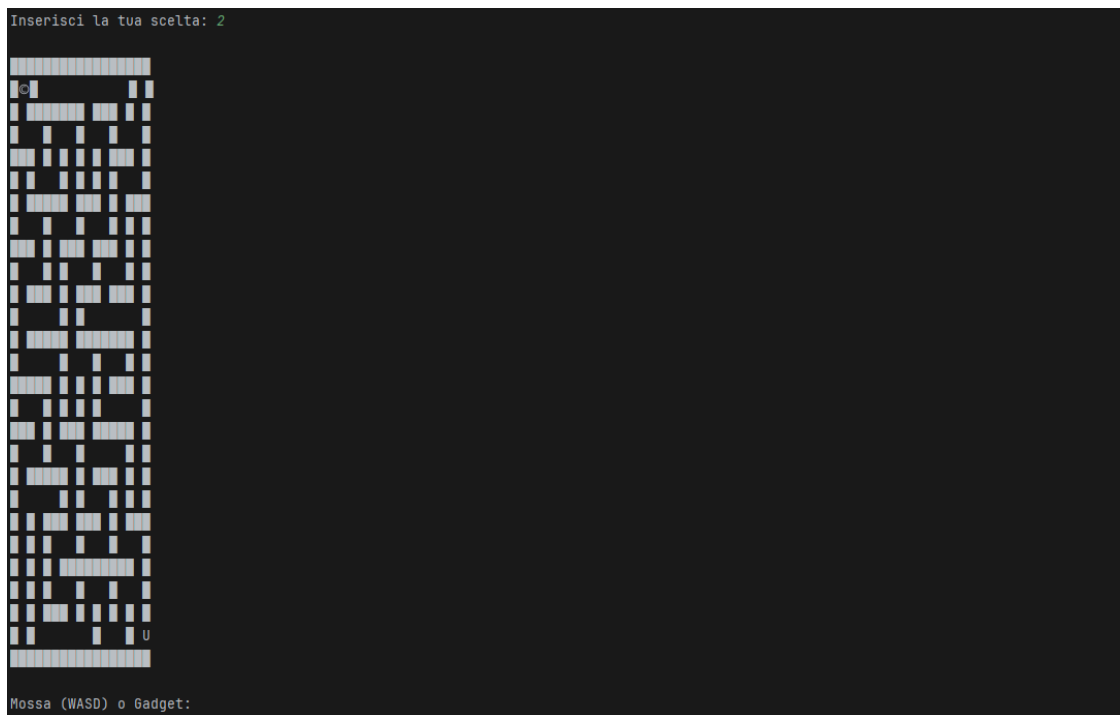
-Menu



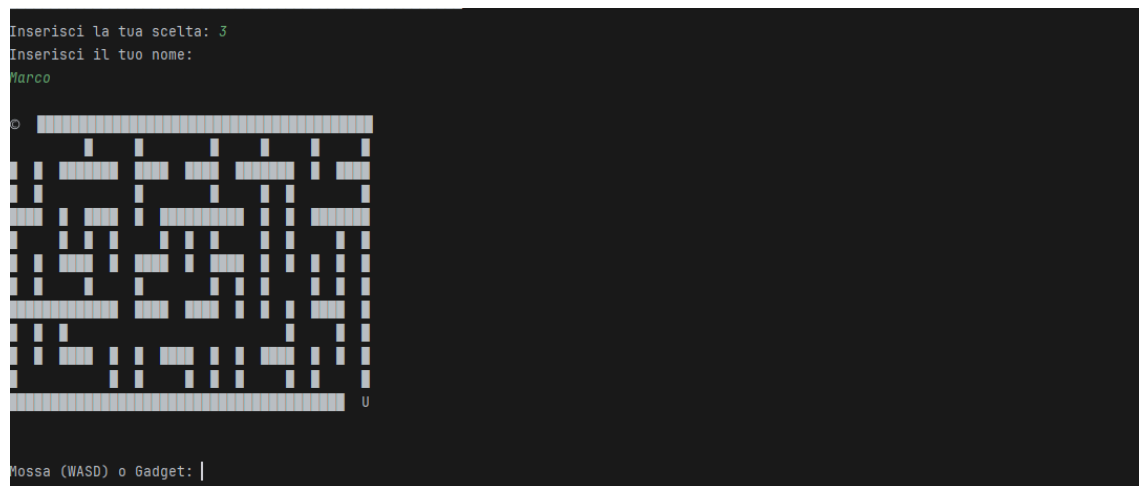
-Modalità storia



-Modalità storia: Gadget



-Modalità allenamento



-Modalità torneo